

Module #6

Variables et Pointeurs.

1. Variable locale, globale et statique.	1
1.1. Les variables locales et globales.	1
1.2. Variables statiques..	2
1.3. Variables globale static.....	3
2. Les pointeurs.....	3
2.1. Déclaration.....	3
2.2. Initialisation	4
2.3. Utilisation	4
2.4. Opérations sur les pointeurs.....	4
2.5. Les pointeurs et les tableaux.....	5
2.6. Quand utiliser les pointeurs? (2 exemples)	6
3. La fonction scanf().....	9
4. Convertisseur ADC	11
4.1. Convertisseurs du STM32F103.....	12
4.2. Résolution du convertisseur	13
4.3. Utilisation	14

1. Variable locale, globale et statique.

C'est ce qu'on va appeler la visibilité de nos variables.

1.1 Les variables locales et globales.

Jusqu'à maintenant, l'ensemble de nos variables était déclarées au début de nos fonctions, tout de suite après l'accolade ouvrante '{'. Cette déclaration donnait le statut de "locales" à nos variables. C'est-à-dire que les variables n'existent qu'à l'intérieur des fonctions dans lesquelles elles sont déclarées.

Variables locales : (déclarées au tout début des fonctions)

Lorsqu'une variable est déclarée dans une fonction (même pour le main), elle est reconnue dans toute cette fonction (**localement**), mais pas dans les autres fonctions. Des variables se trouvant dans des fonctions différentes peuvent porter le même nom. Type de variable à utiliser le plus souvent possible.

Les variables locales sont éphémères (sauf pour celles du main()).

Variables globales : (Déclarées avant le 'main')

Une variable globale est une variable déclarée à l'extérieur de toutes fonctions. Une telle variable est utilisable par l'ensemble des fonctions du programme. Elle est donc connue par chaque fonction du programme qui n'utilise pas une variable locale du même nom (Ce qui peut être Dangereux).

Les variables globales ne sont pas éphémères.

Une bonne pratique est d'utiliser le moins possible les variables GLOBALES.

En fait, à moins d'indications contraire, il est **interdit** d'utiliser les variables globales.

Exemple: Utilisation d'une fonction qui affiche des valeurs locales et globales passées en paramètres

```
void vAffiche(UC ucX, UC ucY);           //

UC ucgA = 1;                            // Variable Globale.
UC ucgB = 2;                            // "

void main(void)                          //
{                                         //
    UC ucC = 3;                          // Variable Locale au main( ).
    UC ucD = 4;                          // "

    vInitPortSerie();                    //
```

```

    vAffiche(ucgA, ucgB);          // Var Globales passées en paramètres.
    vAffiche(ucC, ucD);           // Var Locales passées en paramètres.
    while(1);                      //
}                                  //

void vAffiche(UC ucX, UC ucY)      // 2 variables locales: ucX et ucY.
{                                  //
    printf("%u, %u. ", (int)ucX, (int)ucY); // Affiche les Variables Locales
    printf("%u, %u. ", (int)ucgA, (int)ucgB); // Affiche les Variables Globales
}                                  //

```

Résultat : 1, 2. 1, 2. 3, 4. 1, 2.

Note: Utiliser le même nom de variable locale et globale, peut causer des problèmes de "Shadowing" avec certains compilateurs.

1.2 Variables statiques.

Les variables locales sont déclarées au début des fonctions et sont éphémères. Donc à chaque fois qu'on appelle une fonction, les variables sont à nouveau créées et réinitialisées au besoin. Dans certain cas ça peut être embêtant.

Exemple:

```

UC ucCompteBoite(void)           // Lit Capteur et compte les boites.
{
    UI uiBoite = 0;              // Pour compter les boites.

    if(P3_0 == 0)                // Une boite présente?
    {
        uiBoite++;               // Une boite de plus.
    }
    return uiBoite;
}

```

Malheureusement, cette fonction retournera toujours 1 si une boite est détectée et 0 si aucune boite n'est détectée. Ce n'est pas le but recherché.

La variable uiBoite est créée et remis à 0 à chaque début de la fonction.

Il est possible, avec l'attribut "static" de dire au compilateur que la variable doit continuer d'exister même si la fonction se termine. Ceci nous permettra d'accumuler le nombre de boite.

```

UC ucCompteBoite(void)           // Lit Capteur et compte les boites.
{
    static UI uiBoite = 0;        // Pour compter les boites.
                                   // Initialisée, une fois, au début du programme.
    if(P3_0 == 0)                // Une boite présente?
    {
        uiBoite++;               // Une boite de plus.
    }
    return uiBoite;
}

```

Avec cette déclaration, la variable uiBoite ne sera pas placée sur la pile et ne sera pas éphémère.

Si on rappelle la fonction une deuxième fois la variable uiBoite aura au départ de la fonction sa dernière valeur (elle sera initialisée à 0, seulement, lors du premier passage dans la fonction).

1.3 Variables globale static

Une variable globale déclarée "static", sera globale à l'intérieur du fichier dans lequel elle est déclarée.

Si le projet comporte plusieurs fichiers, les autres fichiers ne pourront voir la variable globale static.

4. Les pointeurs.

Un pointeur est une variable qui contient l'adresse mémoire d'une donnée particulière.

Il pointe l'endroit de la mémoire où est stockée une information.

Un pointeur sera toujours associé à un type de données particulier. Le compilateur doit savoir si le pointeur pointe sur des variables de 1, 2, 4 octets...

Les pointeurs peuvent pointer sur tous les types de données existantes ou définis par le programmeur.

2.1 Déclaration

TypePointé *NomDuPointeur;

Exemple: int *ipPointeur; // Pointe une zone mémoire remisant des entiers.

 char *cpPtr; // Pointe une zone mémoire remisant des octets.

2.2 Initialisation .

L'initialisation d'un pointeur s'effectue en lui affectant la valeur de l'adresse d'une variable déjà existante. Pour y arriver nous utilisons l'opérateur d'adressage &.

Ex.: &ucTotal représente l'adresse où est stockée la valeur de la variable du nom de ucTotal.

Pour initialiser un pointeur il suffit de lui affecter cette valeur.

Ex.:

```

unsigned char ucVal = 12;    // Déclare et initialise la variable ucVal.
unsigned char *ucpPteur;    // Déclare un pointeur du nom de ucpPteur
                             // pointant sur un octet non signé.
ucpPteur = &ucVal;          // Initialise un pointeur du nom de ucpPteur
                             // à l'adresse de la variable ucVal.
                             // ucpPteur pointe sur la variable ucVal.
```

ATTENTION: Un pointeur qui n'est pas initialisé est l'une des sources d'erreurs les plus difficiles à régler.

2.3 Utilisation.

Pour pouvoir manipuler la valeur stockée à l'adresse contenue par le pointeur nous utilisons l'opérateur de déréréfencement noté * (astérisque).

```

*ucpPteur = 0x0d;           // Remise la valeur 0x0d dans la variable ucVal.
ucTotal = *ucpPteur;        // Permet de remiser dans ucTotal la valeur pointée
                             // par le pointeur ucpPteur.
```

* ucpPteur représente alors la valeur stockée à l'adresse indiquée par le pointeur ucpPteur.

4.4 Opérations sur les pointeurs.

Pour incrémenter le pointeur d'une position nous pouvons utiliser l'expression suivante:

```
ipPointeur = ipPointeur + 1; // Pointe l'élément suivant.
```

Si nous voulons décrémenter le pointeur de 5 positions;

```
ipPointeur = ipPointeur - 5; // Pointe cinq éléments avant.
```

NOTE: Ne pas initialiser un pointeur avec un nombre numérique. Ex.: `ipPointeur = 0x2345;`
 Nous ne connaissons pas ce qu'il y a à l'adresse 0x2345.
 Nous laissons au compilateur le soin de nous allouer une location de mémoire disponible.

4.5 Les pointeurs et les tableaux

Du point de vue du type, le nom d'un tableau est considéré comme une constante d'adresse définissant le début de l'enregistrement.

Le nom du tableau représente en fait l'adresse du premier élément. Nous pouvons donc l'associer à une variable pointeur sans avoir à recourir au symbole &.

Exemple:

```
void main(void)
{
    int i = 2;
    int iVal;           // Déclaration d'un entier de nom iVal.
    int iTableau[5];    // d'un vecteur de 5 entiers.
    int *ipPointeur;    // d'un pointeur sur des entiers.

    ipPointeur = iTableau; // Initialisation du pointeur sur le
                          // vecteur du nom de iTableau.
                          // ipPointeur = adresse du tableau.

    *ipPointeur = 8;      // Met 8 dans le premier element 0 du tableau.
    *(ipPointeur+1) = 10; // 10 dans deuxieme element.
    ipPointeur++;         // Pointe le deuxieme element.
    *ipPointeur = 12;     // 12 dans le deuxieme element.
    *(ipPointeur+1) = 87; // 87 dans le troisieme element.
    *(ipPointeur+i) = 45; // 45 dans le quatrieme element.
    i++;
    *(ipPointeur+i) = 68; // 68 dans le cinquieme element.
    i++;
    *(ipPointeur+i) = 18; // Attention! On pointe sur le pointeur!
}
// iTableau contient: {8, 12, 87, 45, 68}
```

001C		ipPointeur
0018		iTableau
0014		
0010		
000C		
0008		iVal
0004		
0000		i

4.6 Quand utiliser les pointeurs? (2 exemples).

Utiliser les pointeurs pour permettre à une fonction de modifier plus d'une variable de la fonction appelante.

```
void vFonctionQuiModifieDeuxVariables(UC *ucpVar1, UC *ucpVar2);

void main(void)
{
    UC ucVariable1 = 'A';
    UC ucVariable2 = 'B';
    vFonctionQuiModifieDeuxVariables(&ucVariable1, &ucVariable2);
    printf("%c%c",ucVariable1, ucVariable2);          // Affiche: OK
    while(1);
}

void vFonctionQuiModifieDeuxVariables(UC *ucpVar1, UC *ucpVar2)
{
    *ucpVar1 = 'O';
    *ucpVar2 = 'K';
}
```

Utiliser les pointeurs pour passer de "gros" éléments à une fonction.

```
void vFonctionModifieTableau(int *ipVar);

void main(void)
{
    int iTab[20] = {0,1,2,3,4,5,6,7,8,9,0,1,2,3,4,5,6,7,8,9}; // 80 octets.

    vFonctionModifieTableau(iTab);    // Passe adresse du tableau (4 octets).
    printf("%d,%d",iTab[10],iTab[11]); // Affiche: 100,121
    while(1);
}

void vFonctionModifieTableau(int *ipVar) // Recoit l'adresse d'un "int".
{
    int i;
    for(i=0; i<20; i++)
    {
        // Il faut que l'adresse reçu, pointe sur bloc
        *(ipVar+i) = i*i;    // d'au moins 20 "int".
    }
}
```

On ne passe jamais un tableau à une fonction. On passe toujours l'adresse du tableau.

Exemple : Accès externe et tableau contenant diverses informations.

```

#define OFF 0
#define ON 1

#define START 0 // Bouton pese = 0.

#define ETATFAN 0
#define VITFANLUE 1

void vControlFan(UC *ucpTab);
void vLireBouton(UC *ucpTab);
void vLireEtat(UC *ucpTab);

void main(void)
{
    UC ucTableau[2] = {OFF,0}; // ETATFAN, VITFANLUE

    while(1)
    {
        vLireBouton(ucTableau); // ou vLireBouton(&ucTableau[0]);
        vLireEtat(ucTableau);
        vControlFan(ucTableau);
    }
}

////////////////////////////////////
void vControlFan(UC *ucpTab)
{
    UC *ucpAdrFan = 0x8300; // CS3.

    if( (*(ucpTab + ETATFAN) == ON) && (*(ucpTab + VITFANLUE) == 0) )
    {
        *ucpAdrFan = 100; // Ventilateur a 100%
    }

    if( (*(ucpTab + ETATFAN) == OFF) && (*(ucpTab + VITFANLUE) != 0) )
    {
        *ucpAdrFan = 0; // Ventilateur a 0%
    }
}

////////////////////////////////////
void vLireBouton(UC *ucpTab)
{
    // Autre méthode:
    if(START) { *(ucpTab + ETATFAN) = ON; } // ucpTab[ETATFAN] = ON;
    else { *(ucpTab + ETATFAN) = OFF; } // ucpTab[ETATFAN] = OFF;
}

////////////////////////////////////
void vLireEtat(UC *ucpTab)
{
    UC xdata *ucpAdrFan = 0x8300; // CS3.

    *(ucpTab + VITFANLUE) = *ucpAdrFan; // Lit vitesse du ventilateur.

    // ucpTab[VITFANLUE] = *ucpAdrFan; // Même chose.
}
    
```


Exercices sur les pointeurs (pour simplifier: "int" et pointeurs à 16 bits)

5. La fonction scanf() :

Maintenant que nous avons vu les pointeurs, il est possible de regarder une nouvelle fonction de lecture au clavier pré-déclarée dans stdio.h.

- La fonction "scanf()" permet de **lire** des valeurs de type **entier**, réel (**float**), et même une **chaîne de caractères** sur l'entrée standard (clavier).
- La lecture d'une variable se termine avec **Enter**, **Espace** ou un caractère non valide.

Le format des arguments ressemble un peu à celui du printf.

Il y a deux sections:

- La première (entre " ") indique le **format** utilisé pour lire les données.
- La seconde donne la **liste des variables** que nous voulons lire.

Comme nous voulons modifier des variables de la fonction appelante, nous devons fournir l'adresse (&) des variables.

Exemple:

```
UI uiVal;
```

```
...
```

```
scanf("%u", &uiVal); // Où &uiVal représente l'adresse de uiVal.
```

Les spécifications de **format** (simplifiées) on la forme suivante :

% «width» «{hh|h|l}» type

-Le champ **type** est un caractère qui spécifie si l'entrée est de type nombre, caractère ou chaîne de caractères, comme l'indique la table suivante (les plus communs):

Character	Argument Type	Input Format
d	int *	Nombre entier signé.
u	unsigned int *	Nombre entier non signé.
x	unsigned int *	Nombre hexadécimal non signé.
f	float *	Nombre point flottant.
c	char *	Un simple caractère. (getchar fait mieux la job)
s	char *	Une chaîne de caractères terminés par "\0".

- Le champ «**width**» est un nombre qui spécifie le maximum de caractère lue sur l'entrée standard.
- Les **options hh, h et l** doivent immédiatement précéder le type pour spécifier **char, short, ou long** pour des entiers de types d, u, et x.
- Un caractère qui suit le symbole "%" et qui n'est pas reconnu comme un format spécifié sera traité comme un caractère ordinaire. Par exemple, "%" scanf va s'attendre à recevoir le symbole "%" en entrée.
- La fonction "scanf" retourne le nombre d'entrée correctement lues. EOF si une erreur est rencontrée.

Exemples de scanf (tiré du manuel du compilateur Keil page 303):

```
#include <stdio.h>
#include "_DeclarationGenerale.h"

void vTestScanf(void);
void vInitPortSerie(void);

void main(void)
{
    vInitPortSerie();
    vTestScanf();
    while(1);
}

void vTestScanf(void)
{
    char a;           // Variables signees.
    int b;
    long c;
    unsigned char x;   // Variables non signees.
    unsigned int y;
    unsigned long z;
    float f, g;        // Variables float.
    char buf [10];     // Variables char.
    int argsread;      // Nombre d'arguments lus.

    printf ("\nEnter a signed byte (Uchar), int, and long: "); // Signees.
    argsread = scanf ("%hhhd %d %ld", &a, &b, &c);

    printf ("\nA= %d, B= %d C= %ld", a, b, c);
    printf ("\n%d arguments read\n", argsread);

    printf ("\nEnter an unsigned byte, int, and long: "); // Non signees.
    argsread = scanf ("%hhu %u %lu", &x, &y, &z);
    printf ("\nX= %u Y= %u Z= %lu", x, y, z);
    printf ("\n%d arguments read\n", argsread);

    printf ("\nEnter a string: "); // Caracteres.
    argsread = scanf (" %9s", buf); // Pour 1 seul Car, utiliser getchar.
    getchar();                     // Pour vider buffer de lecture!

    printf ("\nBuf= %s", buf);
    printf ("\n%d arguments read\n", argsread);

    printf ("\nEnter two floating-point numbers: "); // Floats.
    argsread = scanf ("%f %f", &f, &g);
    printf ("\nF= %f G= %5.2f", f, g);
    printf ("\n%d arguments read\n", argsread);
}
```

4 Convertisseur ADC

Jusqu'à maintenant nous avons activé et lue des éléments exclusivement numériques (0 ou 1). Malheureusement, la vie n'est pas numérique. Elle est plutôt analogique.

C'est pourquoi il existe des circuits permettant de faire le pont entre le monde extérieur (analogique) vers le monde des ordinateurs (numérique).

Le premier que nous allons voir est le circuit qui permet de convertir une tension analogique en signaux numériques pouvant être lus par un ordinateur. Il s'agit du **Convertisseur Analogique Numérique (CAN)** ou en anglais, Analog Digital Converter (**ADC**). La plage de tensions à convertir est différente d'un circuit à un autre, mais elle est souvent ajustable. Dans notre cas nous convertirons une tension qui se situe entre 0 et 5 volts.

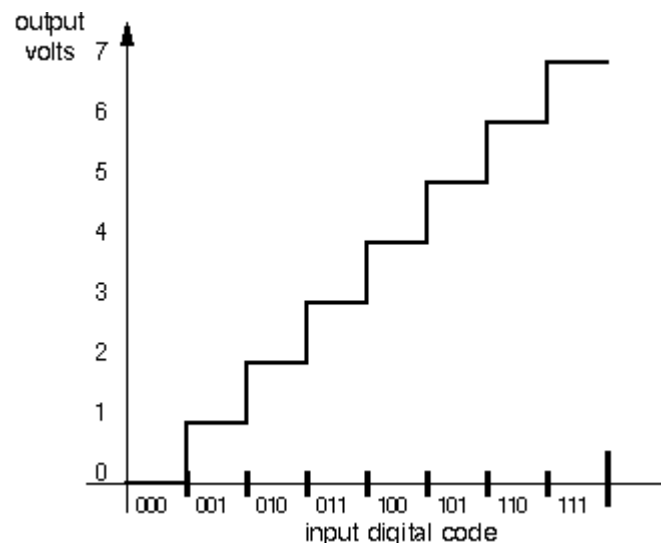
Le principe de la conversion consiste à donner un numéro à chacune des tensions possibles. Comme la tension peut prendre une infinité de valeur, les concepteurs de circuit ont limité le nombre d'états possibles. C'est ce qu'on va appeler la **résolution** du convertisseur. Dans les faits, il s'agit du nombre de bits utilisés pour donner un numéro.

Exemple: Avec 3 bits on peut donner 8 numéros différents. Si la tension est 0 volts on donne le numéro 0 et si la tension est 7 volts on donne le numéro 7. Entre ces deux tensions on donne les numéros 1 à 6.

Le passage d'un niveau à l'autre s'appelle un PAS. Dans notre exemple il y a 7 pas ($2^n - 1$).

La tension maximale de 7 volts est notre tension de référence (V_{ref}).

Dans nos programmes, nous pourrons connaître la tension du signal en multipliant la lecture numérique par $V_{ref} / (2^n - 1)$.



Exemple: J'ai un convertisseur 4 bits. La tension de référence est de 5 volts.
 Mon programme lit 0x0C (12) sur le convertisseur.
 Quel est la tension du signal?

Solution: Tension à l'entrée = $12 * 5V / (2^4 - 1) \rightarrow 60V / 15 \rightarrow 4.0 \text{ volts}$

Il existe plusieurs types de convertisseur analogique à numérique. Ils diffèrent par la technique employée (Simple rampe, Double rampe, Approximations successives, Flash, Semi-flash, Sigma-Delta). Le but du cours n'étant pas de vous décrire l'ensemble de ces techniques, nous allons nous contenter de comprendre l'utilisation du convertisseur du microcontrôleur STM32F103.

4.1 Convertisseurs du STM32F103.

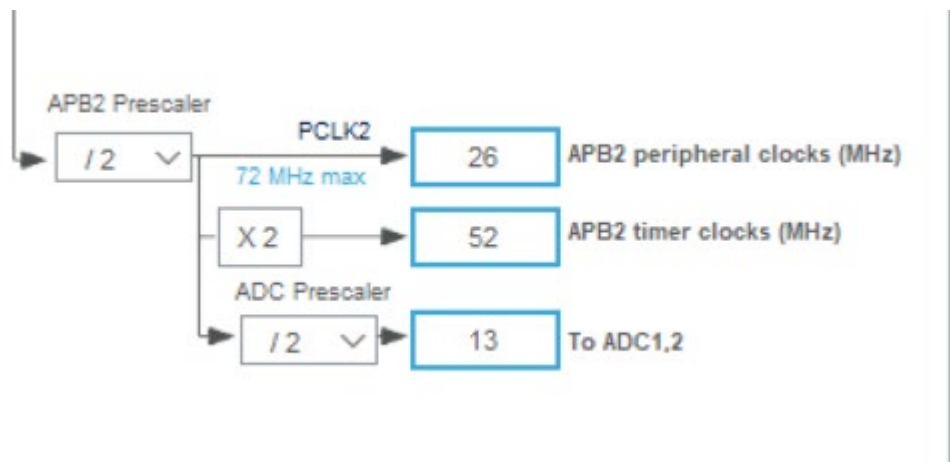
Le CAN **12 bits** est un convertisseur analogique-numérique à approximations successives. Il dispose de 18 canaux multiplexés maximum, lui permettant de mesurer des signaux provenant de seize sources externes et de deux sources internes.

La conversion A/N des différents canaux peut être effectuée en mode simple, continu, par balayage ou discontinu.

Le résultat du CAN est stocké dans un registre de données 16 bits aligné à gauche ou à droite.

La fonction de surveillance analogique (watchdog) permet à l'application de détecter si la tension d'entrée dépasse les seuils haut ou bas définis par l'utilisateur.

L'horloge d'entrée du CAN est générée à partir de l'horloge PCLK2 divisée par un prédiviseur (prescaler) et ne doit pas dépasser 14 MHz.



4.2 Résolution du convertisseur.

La **résolution** de l'ADC est le nombre de **niveaux de quantification** d'un signal analogique

Le **STM32F103** possède un **ADC 12 bits**, ce qui signifie :

- Résolution : $2^{12} = 4096$ niveaux
- Donc, la tension analogique mesurée est divisée en **4096 pas**.

Taille d'un pas (LSB)

Si la tension de référence est de **3.3 V**, alors chaque pas (ou LSB = Least Significant Bit) vaut :

$$\text{LSB} = \frac{V_{\text{ref}}}{2^{12}} = \frac{3.3 \text{ V}}{4096} \approx 0.000805 \text{ V} = 0.805 \text{ mV}$$

Un pas représente environ 0.805 mV.

La **valeur numérique** de l'ADC va de **0** (pour 0 V) à **4095** (pour 3.3 V)

Exemple :

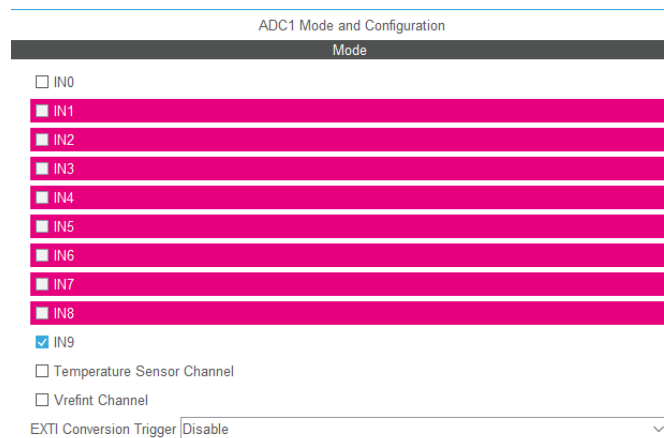
Si tu lis une tension de 1.65 V (la moitié de 3.3 V), tu obtiens :

$$\text{Valeur ADC} = 1.65/3.3 \times 4095 \approx 2047$$

4.3 Utilisation

Configuration dans le STM32CubeMX.

L'activation d'un canal de lecture ADC se fait à partir de la fenêtre « Mode and Configuration » des périphériques, par le choix du convertisseur et l'attribution d'un canal.



Dans l'exemple illustré ci-haut, le choix s'est arrêté sur le canal IN9 correspondant à la broche PB1 du MCU pour le convertisseur intégré ADC1.

Il faut au préalable, configurer la broche du MCU en mode ADC1.

Utilisez les fonctions de l'API HAL_ADC. En mode « Pooling », les fonctions nécessaires à l'usage du convertisseur seront :

- HAL_ADC_Start()
- HAL_ADC_Stop()
- HAL_ADC_PollForConversion()
- HAL_ADC_GetValue()

Et le paramètre principal sera le « handle » du convertisseur, soit le hadc1.

Dans l'exemple, l'utilisation des fonctions de l'API de l'ADC pourrait se faire comme suit.

```
HAL_ADC_Start(&hadc1);           //Démarrer la conversion.
HAL_ADC_PollForConversion(&hadc1,10 ); //Attendre la fin de la conversion.
uiValeurADC1 = HAL_ADC_GetValue(&hadc1); //Lire la donnée.
HAL_ADC_Stop(&hadc1);           //Arrêter la conversion.
```

Exemple de lectures à intervalle régulier:

```
UI uiData[2000];                // Tableau des lecture.
UI i;

for (i = 0; i < 2000; i++)      // Remplit tableau (Temps = Intervalle * 2000).
{
    if(ucFlagTM2 == 1)          // Intervalle passer, on fait une lecture.
    {
        ucgFlagTM2 = 0;        // Remet Flag à 0.
        HAL_ADC_Start(&hadc1); //Démarrer la conversion.

        if (HAL_ADC_PollForConversion(&hadc1, 5) == HAL_OK) // Attend la fin de la
                                                                // conversion
        {
            ucData[i] = HAL_ADC_GetValue(&hadc1); //Lire la donnée;
        }
    }
    Else
    {
        i--;                    // Pas de conversion, donc on reste au même "i"
    }
}
```